

Software Testing

Automated Testing

João Saraiva ★, Marcos Viera ★★

2025/2026

★*Departamento de Informática*
Universidade do Minho, Portugal

★★*Universidade de Montevideo*
Uruguai

Automated Testing

**QuickChec
k**

Automated Testing - QuickCheck

QuickCheck is an excellent Haskell library that allows expressing **properties about programs** and testing them with **randomly-generated values**.

Automated Testing - QuickCheck

QuickCheck

From Wikipedia, the free encyclopedia

For the convenience store, see [Quick Chek](#).

QuickCheck is a [software library](#), specifically a [combinator library](#), originally written in the programming language [Haskell](#), designed to assist in [software testing](#) by generating [test cases](#) for [test suites](#).

Contents [\[hide\]](#)

- [Software](#)
- [See also](#)
- [References](#)
- [Further reading](#)

Software [\[edit\]](#)

It is compatible with the [compiler](#), [Glasgow Haskell Compiler](#) (GHC) and the [interpreter](#), Haskell User's Gofer System ([Hugs](#)). It is [free and open-source software](#) released under a [BSD-style](#) license.

In QuickCheck, assertions are written about logical properties that a [function](#) should fulfill. Then QuickCheck attempts to generate a test case that [falsifies](#) such assertions. Once such a test case is found, QuickCheck tries to reduce it to a minimal failing subset by removing or simplifying input data that are unneeded to make the test fail.

The project began in 1999. Besides being used to test regular programs, QuickCheck is also useful for building up a [functional specification](#), for documenting what functions should be doing, and for testing compiler implementations.^{[\[1\]](#)}

Re-implementations of QuickCheck exist for several languages:

- [C](#)^{[\[2\]](#)[\[3\]](#)[\[4\]](#)}
- [C++](#)^{[\[5\]](#)[\[6\]](#)[\[7\]](#)}
- [Chicken](#)^{[\[8\]](#)}
- [Clojure](#)^{[\[9\]](#)[\[10\]](#)[\[11\]](#)}
- [Common Lisp](#)^{[\[12\]](#)}
- [Coq](#)^{[\[13\]](#)}
- [D](#)^{[\[14\]](#)}
- [Elm](#)^{[\[15\]](#)}
- [Elixir](#)^{[\[16\]](#)}
- [Erlang](#)^{[\[17\]](#)}
- [F#](#), and [C#](#), [Visual Basic .NET](#) ([VB.NET](#))^{[\[18\]](#)}
- [Factor](#)^{[\[19\]](#)}
- [Go](#)^{[\[20\]](#)}
- [Io](#)^{[\[21\]](#)}
- [Java](#)^{[\[22\]](#)[\[23\]](#)[\[24\]](#)[\[25\]](#)[\[26\]](#)[\[27\]](#)[\[28\]](#)}
- [JavaScript](#)^{[\[29\]](#)[\[30\]](#)[\[31\]](#)[\[32\]](#)}
- [Julia](#)^{[\[33\]](#)}
- [Logtalk](#)^{[\[34\]](#)}
- [Lua](#)^{[\[35\]](#)}
- [Objective-C](#)^{[\[36\]](#)}
- [OCaml](#)^{[\[37\]](#)}
- [Perl](#)^{[\[38\]](#)}
- [Prolog](#)^{[\[39\]](#)[\[40\]](#)}
- [PHP](#)^{[\[41\]](#)}
- [Pony](#)^{[\[42\]](#)}
- [Python](#)^{[\[43\]](#)}
- [R](#)^{[\[44\]](#)}
- [Racket](#)^{[\[45\]](#)}
- [Ruby](#)^{[\[46\]](#)}
- [Rust](#)^{[\[47\]](#)[\[48\]](#)}
- [Scala](#)^{[\[49\]](#)[\[50\]](#)[\[51\]](#)}
- [Scheme](#)^{[\[52\]](#)}
- [Smalltalk](#)^{[\[53\]](#)}
- [Standard ML](#)^{[\[54\]](#)}
- [Swift](#)^{[\[55\]](#)}
- [TypeScript](#)^{[\[56\]](#)}
- [Whiley](#)^{[\[57\]](#)}

QuickCheck

Developer(s)	Koen Claessen, John Hughes
Initial release	1999; 21 years ago
Stable release	2.6 / 7 March 2013; 7 years ago
Written in	Haskell
Operating system	Unix-like, Windows
Available in	English
Type	Software testing
License	BSD-style
Website	www.cse.chalmers.se/~rjmh/QuickCheck/

Automated Testing - QuickCheck

How does it work?

Suppose we have a program we want to test

Example:

```
import Test.QuickCheck
```

```
reverse [] = []  
reverse (x:xs) = reverse xs ++ [x]
```

Automated Testing - QuickCheck

We define a **property** as Haskell function that returns a boolean.

Example:

```
prop_RevApp :: [Int] -> [Int] -> Bool
prop_RevApp xs ys = reverse (xs++ys) ==
                    reverse ys ++ reverse xs
```

(Convention: Properties usually start with *prop_*)

Automated Testing - QuickCheck

We apply the function **quickCheck** to test the property.

By default it uses 100 **randomly-generated values**.

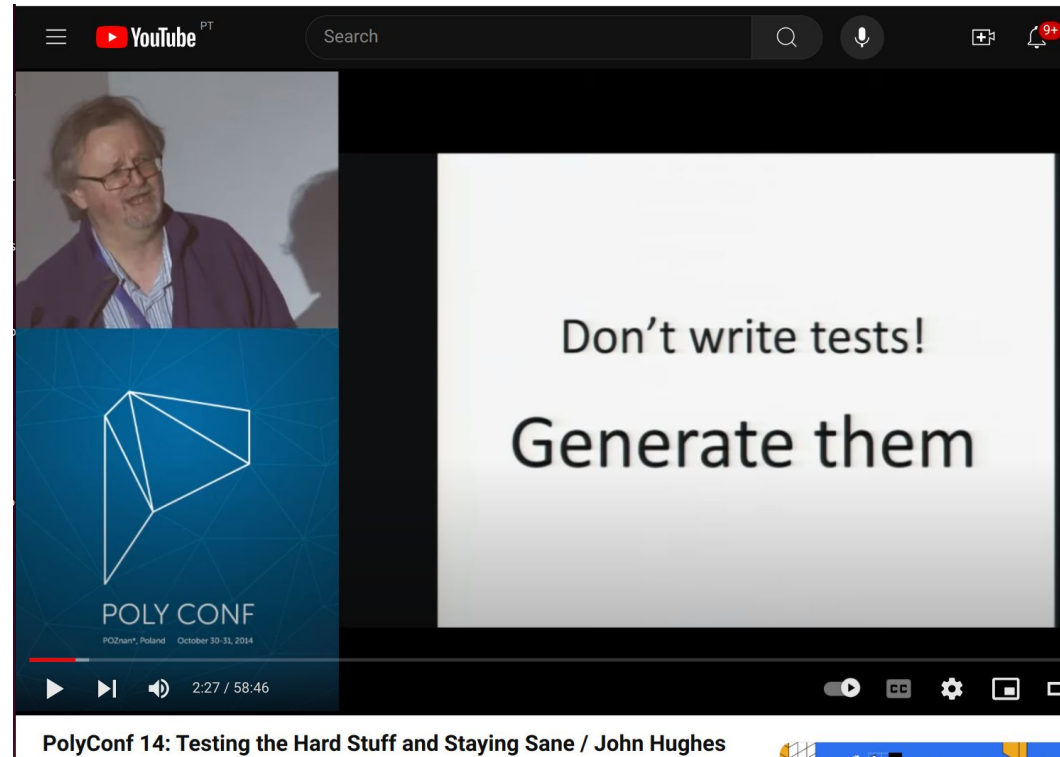
Example Running QuickCheck:

```
*Main> quickCheck prop_RevApp  
+++ OK, passed 100 tests.
```

Automated Testing

Random Test Case Generators

Don't write tests! Generate them



[John Hughes - Don't Write Tests!](#)

QuickCheck - Generators

- QuickCheck includes **basic generators** for pre-defined types
- and **combinators** for making new ones.
- **Gen a**: generator of values of type **a**
 - **Gen** is a **monad** that combines:
 - A pseudo random generator seed, and
 - a size measure for data structures (i.e. list size)

QuickCheck - Generators

QuickCheck provides the **Arbitrary type class** for random generation and shrinking values:

```
class Arbitrary a where
  arbitrary :: Gen a
  shrink    :: a -> [a]
```

And instances for most types in base: `()`, `Bool`, `Int`, `Integer`, `Float`, `Double`, `Char`, `[a]`, `Maybe a`, **etc.**

We can print some examples of the generated values using:

```
sample :: Show a => Gen a -> IO ()
```

QuickCheck - Combinators

```
choose      :: Random a => (a, a) -> Gen a
oneof       :: [Gen a] -> Gen a
frequency   :: [(Int, Gen a)] -> Gen a
elements    :: [a] -> Gen a
shuffle     :: [a] -> Gen [a]
suchThat    :: Gen a -> (a -> Bool) -> Gen a
resize      :: Int -> Gen a -> Gen a
sized       :: (Int -> Gen a) -> Gen a
```

QuickCheck - Basic Generators

- A simple generator for Cardinal Points

```
data Cardinal = North | East | South | West

genCardinal :: Gen Cardinal
genCardinal = elements [North, East, South, West]
```

```
*Main> sample genCardinal
East
East
North
East
West
South
```

QuickCheck - Basic Generators

- A simple generator for Cardinal Points

```
data Cardinal = North | East | South | West

genCardinal :: Gen Cardinal
genCardinal = elements [North, East, South, West]
```

We can define an instance of Arbitrary

```
instance Arbitrary Cardinal where
    arbitrary = genCardinal
```

QuickCheck - Basic Generators

- A generator for dates

```
genDate :: Gen (Int,Int,Int)
genDate = do d <- elements [1..31]
             m <- elements [1..12]
             y <- choose (1900, 2016)
             return (d,m,y)
```

```
*Main> sample genDate
(24,8,1931)
(19,12,1955)
(3,7,1961)
(26,4,1968)
```

QuickCheck - Basic Generators

- Let us consider that we wish to generate “more” dates in the first semester than in the second.

```
genFstSem = elements [1..6]
```

```
genSndSem = elements [7..12]
```

```
genFDate :: Gen (Int,Int,Int)
```

```
genFDate = do d <- elements [1..31]
```

```
    m <- frequency [(3,genFstSem)  
                    , (1,genSndSem)]
```

```
    y <- choose (1900, 2016)
```

```
    return (d,m,y)
```

3/4 in the first semester, and 1/4 in the second one.

QuickCheck - Basic Generators

- . Permutations of the list of lower vowels

```
*Main> sample $ shuffle "aeiou"  
"oaiue"  
"auioe"  
"uaeoï"  
"uaoie"
```

- . Lower and capital vowels, equally probable

```
*Main> sample $ oneof [elements "aeiou"  
                        ,elements "AEIOU"]  
'A'  
'o'  
'A'  
'i'
```

QuickCheck - Basic Generators

Using arbitrary

```
data Age      = Age Int
data Person   = Person String Age

instance Arbitrary Age where
  arbitrary = do age <- choose (0,100)
               return (Age age)

instance Arbitrary Person where
  arbitrary = do n <- elements ["John","Bob"]
               a <- arbitrary
               return (Person n a)
```

QuickCheck - Basic Generators

Polymorphism

```
data Pair a b = Pair a b

instance (Arbitrary a, Arbitrary b)
  => Arbitrary Pair a b where
  arbitrary = do x <- arbitrary
                y <- arbitrary
                return (Pair x y)
```

```
> sample (arbitrary :: Gen (Pair Int Person))
Pair 0 (Person "Bob" (Age 73))
Pair 6 (Person "John" (Age 11))
...
```

QuickCheck - *MyString*

Let us consider again the strings of type *MyString*

```
type MyString = [ (Char,Int) ]
```

In order to define a generator for *MyString* it is useful to express it as a data type (or newtype) so we can create instances of class *Arbitrary*.

```
data MyString = MyString [ (Char,Int) ]
```

And we write “baga” as

```
baga :: MyString
```

```
baga = MyString [ ( 'a' , 3 ) , ( 'g' , 4 ) , ( 'r' , 2 )  
                  , ( 'B' , 1 ) , ( 'a' , 5 ) ]
```

QuickCheck - MyString

We adapt or conversion functions to work on this data type

```
toString :: MyString -> String
toString (MyString l) = map fst(sortOn snd l)

fromString :: String -> MyString
fromString s = MyString $ zip s [1..]
```

QuickCheck - *MyString*

We write a property to test the composition of these functions

```
prop_toFrom :: MyString -> Bool
prop_toFrom l = fromString (toString l) == l
```

- How can we verify this property?

QuickCheck needs to produce random values of type *MyString*!

QuickCheck - *MyString*

Thus, we need to define a generator!

```
genMyString :: Gen MyString
genMyString = do s    <- arbitrary
                  let l = zip s [1..]
                  return (MyString l)
```

And, we need to make MyString an instance of Arbitrary

```
instance Arbitrary MyString where
  arbitrary = genMyString
```

QuickCheck - *MyString*

```
ghci> quickCheck prop_toFrom  
+++ OK, passed 100 tests.
```

- Hum... Should this property hold for all strings of type *MyString*?

QuickCheck - *MyString*

A better generator!

```
genMyString :: Gen MyString
genMyString = do s      <- arbitrary
                  let l = zip s [1..]
                  l'     <- shuffle l
                  return (MyString l')
```

- Should this property hold for all strings of type *MyString*?

```
ghci> quickCheck prop_toFrom
*** Failed! Falsified (after 5 tests):
[('/',2),('?',1)]
```

QuickCheck - *MyString*

We need to produce better counter examples...

```
instance Arbitrary MyString where
  arbitrary = genMyString
  shrink (MyString l) = map MyString (shrink l)
```

```
ghci> quickCheck prop_toFrom
*** Failed! Falsified (after 6 tests and 3
shrinks) :
[ ('a', 0) ]
```

QuickCheck - List Generators

- List Generators

```
listOf    :: Gen a -> Gen [a]  -- random length
listOf1   :: Gen a -> Gen [a]
           -- non-empty lists of random length
vectorOf  :: Int -> Gen a -> Gen [a]
           -- list of the given length
```

```
*Main> sample $ listOf date
[]
[(18,6,1993) , (5,2,1947) ]
[(24,2,1949) , (18,12,1996) , (16,4,1976) ]
[(20,11,1983) , (22,4,1983) ]
```

QuickCheck - List Generators

- List Generators

```
listOf    :: Gen a -> Gen [a] -- random length
listOf1   :: Gen a -> Gen [a]
           -- non-empty lists of random length
vectorOf  :: Int -> Gen a -> Gen [a]
           -- lists of the given length
```

```
*Main> sample (vectorOf 5 $ elements "aeiou")
"uoEOa"
"oieau"
"iaoiu"
"eaaeu"
```

QuickCheck - List Generators

- List Generators

```
orderedList :: (Ord a, Arbitrary a) => Gen[a]
```

```
*Main> sample (orderedList :: Gen [Int])  
[]  
[-2]  
[-3,-2,2]  
[-4,4,4]  
[-6,0,1,2,5]  
[-10,0,1,1,2,2,3,9]
```

QuickCheck - User-defined List Generators

```
data List a = Nil | Cons a (List a)!
```

```
genCons :: Arbitrary a => Gen (List a)
```

```
genCons = do v <- arbitrary  
             l <- arbitrary  
             return (Cons v l)
```

```
GenNil = return Nil
```

```
instance Arbitrary a => Arbitrary (List a) where  
    arbitrary = oneof [genCons, genNil]
```

Mean length is one!

QuickCheck - User-defined List Generators

We can parameterize the generator with the size of the list.

```
genSizedList :: Arbitrary a => Int -> Gen (List a)
genSizedList 0 = return Nil
genSizedList n = do v <- arbitrary
                    l <- genSizedList (n-1)
                    return (Cons v l)
```

QuickCheck - User-defined List Generators

Now we have to define the instance of class Arbitrary....

```
instance Arbitrary a => Arbitrary (List a) where  
  arbitrary = genSizedList ??
```

Hum.... we need to provide the size of the list!

```
instance Arbitrary a => Arbitrary (List a) where  
  arbitrary = genSizedList 5
```

Which produces arbitrary lists... But all with **length 5!!**

QuickCheck - Sized Generators

QuickCheck begins by generating “small” test cases, gradually increasing the **size**. The meaning of **size** is data-dependent:

- ignored for “fixed size” types (e.g. Bool or Char);

- is the length for lists

- is the magnitude for Int and Integer

- ...

The basic combinator can use **sized** to access the size measure in our own generators

```
sized :: (Int -> Gen a) -> Gen a
```

=> *useful for generating recursive data structures*

QuickCheck - User-defined List Generators

Thus, a proper instance for lists would be:

```
instance Arbitrary a => Arbitrary (List a) where  
  arbitrary = sized $ genSizedList
```

```
genSizedList :: Arbitrary a => Int -> Gen (List a)
```

```
genSizedList 0 = return Nil
```

```
genSizedList n = do v <- arbitrary  
                   l <- genSizedList (n-1)  
                   return (Cons v l)
```

QuickCheck - Sorted List Generators

Sorted lists (**in the wrong way**)

```
sortedList :: (Arbitrary a, Ord a) => Gen [a]
sortedList = arbitrary `suchThat` sorted
```

Mainly small randomly generated lists are sorted:

```
*Main> sample (sortedList :: Gen [Int])
[]
[-1]
[6]
[-5, 6]
[]
```

QuickCheck - Sorted List Generators

A better option is to generate lists and then sort them:

```
sortedList :: (Arbitrary a, Ord a) => Gen [a]
sortedList = do l <- arbitrary
               return $ sort l
```

```
*Main> sample $ (sortedList::Gen [Int])
[]
[]
[6]
[-6,5,6]
[-1,8]
[-12,-6,-3]
```

QuickCheck - Sorted List Generators

Or just generating them sorted:

```
sortedList :: ( Arbitrary a
                , Enum a , Ord a --needed by succ
                , Bounded a -- needed by minBound
                , Random a  -- needed by choose
                ) => Gen [a]

sortedList = sized $ genSList minBound maxBound

genSList lo hi 0 = return []
genSList lo hi n
    = do v <- choose (lo,hi)
        l <- genSList (succ v) hi (n-1)
        return $ v : l
```

QuickCheck - Tree Generators

```
data BTree a = Node a (BTree a) (BTree a)
              | Leaf
```

```
genBTree :: Arbitrary a => Gen (BTree a)
genBTree = frequency [(2, genNode), (1, genLeaf)]
```

```
genNode :: Arbitrary a => Gen (BTree a)
```

```
genNode = do  v <- arbitrary
              l <- genBTree
              r <- genBTree
              return (Node v l r)
```

```
genLeaf = return Leaf
```

Non-termination!

QuickCheck - Tree Generators

```
instance Arbitrary a =>Arbitrary (BTree a)where  
  arbitrary = sized $ genBTree
```

```
genBTree 0 = genLeaf
```

```
genBTree n = frequency [(1,genLeaf)  
                        ,(4,genNode n)]
```

```
genLeaf _ = return Leaf
```

```
genNode n = do v <- arbitrary  
               l <- genBTree (n `div` 2)  
               r <- genBTree (n `div` 2)  
               return (Node v l r)
```

QuickCheck - Tree Generators

```
*Main>sample (arbitrary :: Gen (BTree Int))  
Leaf
```

```
Node 2 (Node (-2) Leaf Leaf) (Node 1 Leaf  
Leaf)
```

```
Leaf
```

```
Node 4 (Node 5 (Node (-4) Leaf Leaf) (Node 4  
Leaf Leaf)) (Node (-2) (Node (-4) Leaf Leaf)  
(Node 0 Leaf Leaf))
```

```
Node (-5) Leaf (Node 6 Leaf (Node 2 Leaf (Node  
(-5) Leaf Leaf)))
```

QuickCheck - Tree Generators

```
data Expr = Add    Expr Expr | Mul    Expr Expr
          | Const Float
```

```
instance Arbitrary Expr where
```

```
    arbitrary = sized genExpr
```

```
genExpr s = frequency [(1, do f <- arbitrary
                                return (Const f))
                        , (s, do l <- genExpr s'
                                r <- genExpr s'
                                return (Add l r))
                        , (s, do l <- genExpr s'
                                r <- genExpr s'
                                return (Mul l r))] ]
```

```
    where s' = div s 2
```

QuickCheck - Using Context Information

Let us consider that we wish to generate a list of persons and to associate to each person a unique identifier (for example, its id number)

```
type IdPersons a = [(a, Person)]
```

The **sized** generator gets as input the **list of available keys**:

```
genIdPersons _ 0 = return []
genIdPersons keys n =
    do k      <- elements keys
       let keys' = delete k keys
       ps      <- genIdPersons keys' (n-1)
       person   <- arbitrary
       let p    = (k, person)
       return (p : ps)
```

QuickCheck - Using Context Information

We define now a generator for Persons where the keys are integer numbers.

```
data Persons = Persons (IdPerson Int)

genPersons keys n = do ps <- genIdPersons keys n
                      return (Persons ps)

instance Arbitrary Persons where
  arbitrary = sized $ genPersons [1..1000]
```

To be continued...